DESIGNING DATA INTENSIVE APPLICATIONS

LAB01: TPCH Data Warehouse Lab

1. Preparing the OLTP Database

The first step was preparing the **transactional database (OLTP)** using the TPCH dataset.

The OLTP schema is highly normalized (3NF) and optimized for transactional workloads rather than analytical queries.

Steps

1. Install MySQL.
2. Create a TPCH database.

```
CREATE DATABASE tpch;
USE tpch;
```

3. Import the TPCH tables:

- customer
- orders
- lineitem
- supplier
- nation
- region
- part
- partsupp

These tables represent the **normalized transactional schema (3NF)**.

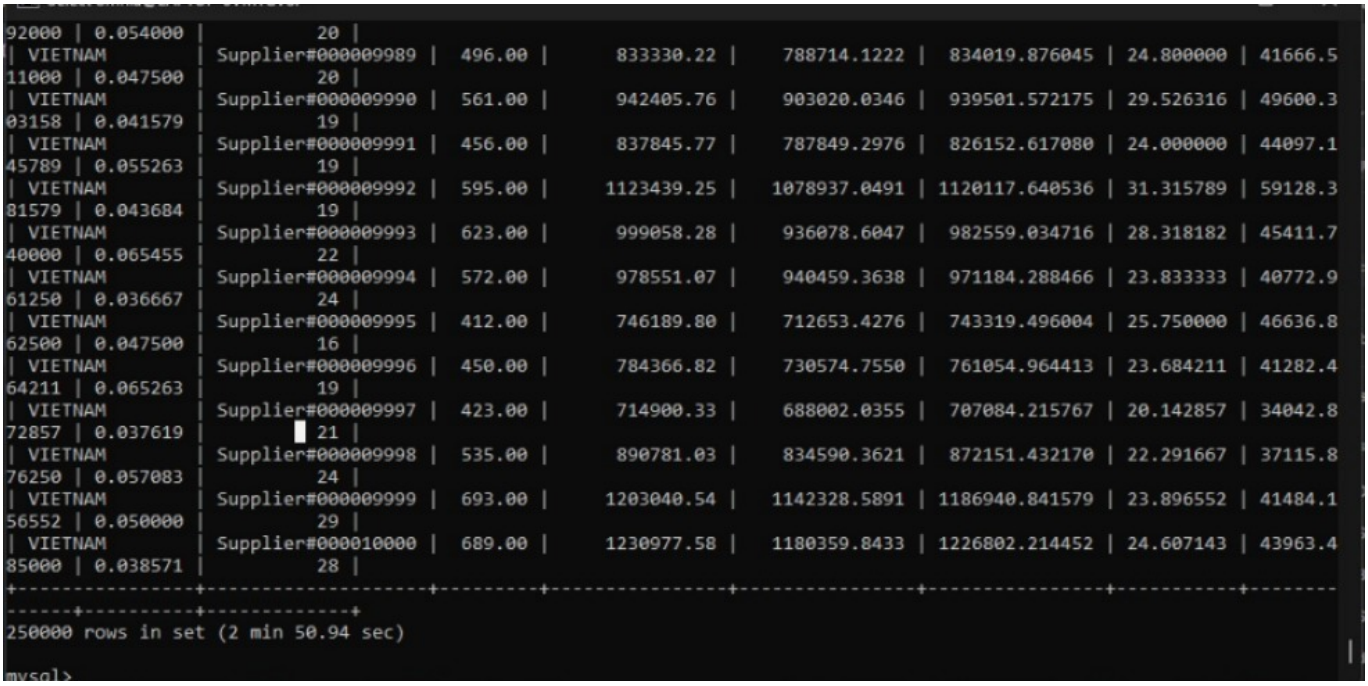
---

2. Query Analysis

Before designing the data warehouse, the analytical query provided in the lab was analyzed.

```
select
    n.N_NAME as n_name,
    s.S_NAME as s_name,
    sum(l.l_quantity) as sum_qty,
    sum(l.l_extendedprice) as sum_base_price,
    sum(l.l_extendedprice * (1 - l.l_discount)) as sum_disc_price,
    sum(l.l_extendedprice * (1 - l.l_discount) * (1 + l.l_tax)) as
```

```
sum_charge,
    avg(l.l_quantity) as avg_qty,
    avg(l.l_extendedprice) as avg_price,
    avg(l.l_discount) as avg_disc,
    count(*) as count_order
from
    lineitem l
    join orders o on l.l_orderkey = o.o_orderkey
    join customer c on o.o_custkey = c.c_custkey
    join nation n on c.c_nationkey = n.n_nationkey
    join partsupp ps on l.l_partkey = ps.ps_partkey and l.L_SUPPKEY =
ps.ps_suppkey
    join supplier s on ps.ps_suppkey = s.s_suppkey
where
    l_shipdate <= date '1998-12-01' - interval '90' day
group by
    n.N_NAME, s.S_NAME;
```



```
92000 | 0.054000 |          20 |
| VIETNAM        | Supplier#000009989 |  496.00 |      833330.22 |    788714.1222 |  834019.876045 | 24.800000 | 41666.5
11000 | 0.047500 |          20 |
| VIETNAM        | Supplier#000009990 |  561.00 |      942405.76 |    903020.0346 |  939501.572175 | 29.526316 | 49600.3
03158 | 0.041579 |          19 |
| VIETNAM        | Supplier#000009991 |  456.00 |      837845.77 |    787849.2976 |  826152.617080 | 24.000000 | 44097.1
45789 | 0.055263 |          19 |
| VIETNAM        | Supplier#000009992 |  595.00 |     1123439.25 |   1078937.0491 | 1120117.640536 | 31.315789 | 59128.3
81579 | 0.043684 |          19 |
| VIETNAM        | Supplier#000009993 |  623.00 |      999058.28 |    936078.6047 |  982559.034716 | 28.318182 | 45411.7
40000 | 0.065455 |          22 |
| VIETNAM        | Supplier#000009994 |  572.00 |      978551.07 |    940459.3638 |  971184.288466 | 23.833333 | 40772.9
61250 | 0.036667 |          24 |
| VIETNAM        | Supplier#000009995 |  412.00 |      746189.80 |    712653.4276 |  743319.496004 | 25.750000 | 46636.8
62500 | 0.047500 |          16 |
| VIETNAM        | Supplier#000009996 |  450.00 |      784366.82 |    730574.7550 |  761054.964413 | 23.684211 | 41282.4
64211 | 0.065263 |          19 |
| VIETNAM        | Supplier#000009997 |  423.00 |      714900.33 |    688002.0355 |  707084.215767 | 20.142857 | 34042.8
72857 | 0.037619 |          21 |
| VIETNAM        | Supplier#000009998 |  535.00 |      890781.03 |    834590.3621 |  872151.432170 | 22.291667 | 37115.8
76250 | 0.057083 |          24 |
| VIETNAM        | Supplier#000009999 |  693.00 |     1203040.54 |   1142328.5891 | 1186940.841579 | 23.896552 | 41484.1
56552 | 0.050000 |          29 |
| VIETNAM        | Supplier#000010000 |  689.00 |     1230977.58 |   1180359.8433 | 1226802.214452 | 24.607143 | 43963.4
85000 | 0.038571 |          28 |
+----------------+--------------------+---------+----------------+----------------+----------------+-----------+--------
------+----------+-------------+
250000 rows in set (2 min 50.94 sec)

mysql>
```

Run	OLTP (min)
Run 1	2 min 50.94 sec
Run 2	3 min 11.49 sec
Run 3	2 min 39.52 sec
Run 4	2 min 55.83 sec
Run 5	2 min 42.87 sec
Run 6	3 min 44.34 sec
Run 7	3 min 28.06 sec
Run 8	3 min 50.30 sec

Run	OLTP (min)
Average	3 min 10.42 sec

Problems:

```
    -> group by
    ->     n.N_NAME, s.S_NAME;

ERROR 3 (HY000): Error writing file '/tmp/MYfd=230' (OS errno 28 - No space left on device)
mysql>
mysql> SELECT COUNT(*) FROM LINEITEM;
+----------+
```

Solution:

Optimization done in WSL for speedup

In mysqld configuration file added :

After -> [mysqld]

ADDED:

Local_infile=1

innodb_redo_log_capacity = 4294967296

innodb_buffer_pool_size = 4G

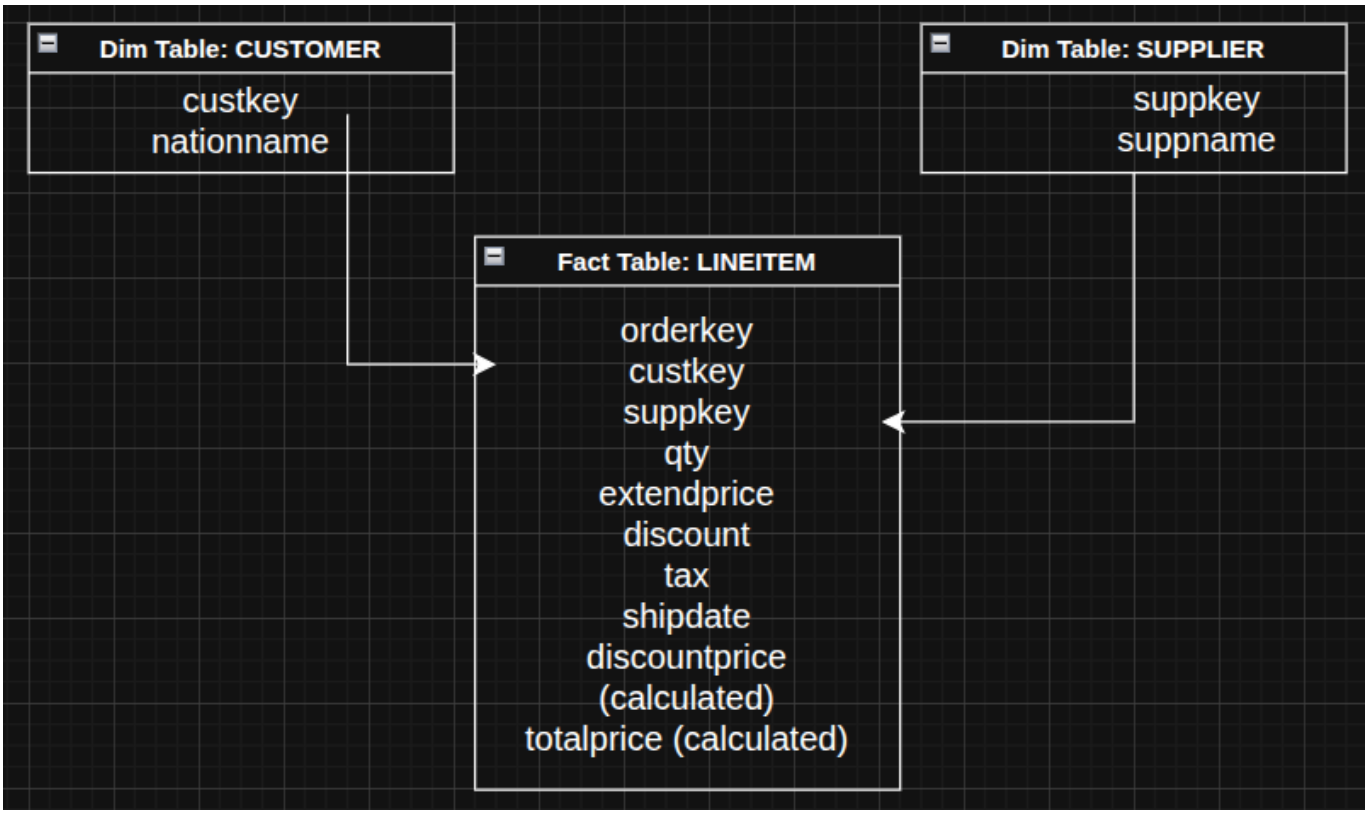
innodb_flush_log_at_trx_commit = 2

innodb_doublewrite = 0

3. Designing the Star Schema

For OLAP workloads, a **Star Schema** was designed.

A star schema improves performance for analytical queries by reducing the number of joins required.



---

4. Designing the ETL Process

The ETL process converts the **OLTP schema into the star schema**. It reads from MySQL, performs the transformation into star schema, and loads the data into parquet files.

ETL stages:

Extract → Transform → Load

Extract

Data is extracted from the OLTP MySQL tables. This stage only involves fetching the data from the source tables and providing input for the transformation stage.

Transform

The data is converted into a target structure appropriate for OLAP supported schemas, which is the Star Schema here. Some operations to achieve this transformation include:

- Table joins
- Filter on rows
- Denormalizing normalized schemas

Load

The transformed data is stored in **Parquet format**.

Parquet was chosen because:

- Columnar storage
- High compression
- Optimized for analytical engines (Spark OLAP queries)

---

5. Installing the ETL Tool

The ETL pipeline was implemented using Apache NiFi.

NiFi provides a visual interface to create and manage data pipelines.

Installation Steps

1. Download NiFi 2.8.
2. Extract the archive.

Example installation directory:

```
/opt/nifi/nifi
```

3. Start NiFi:

```
./bin/nifi.sh start
```

4. Open the NiFi user interface:

```
http://localhost:8443/nifi
```

---

6. Installing Required NiFi Extensions

To support Parquet processing, the following extensions were installed:

- nifi-parquet-nar
- nifi-hadoop-libraries-nar

These files were placed inside the extensions directory:

```
/opt/nifi/nifi/extensions/
```

After adding the files, NiFi was restarted.

---

7. Connecting NiFi to MySQL

To connect NiFi to the MySQL database, a JDBC driver was installed.

Driver used:

```
mysql-connector-j-9.6.0.jar
```

The driver file was placed inside:

```
/opt/nifi/nifi/lib
```

Configuring DBCPConnectionPool

A **DBCPConnectionPool controller service** was configured in NiFi.

Configuration:

Database Connection URL

```
jdbc:mysql://localhost:3306/tpch
```

Driver Class Name

```
com.mysql.cj.jdbc.Driver
```

Driver Location

```
/opt/nifi/nifi/lib/mysql-connector-j-9.6.0.jar
```

Username

```
root
```

```
```

Password

```
<mysql password>
```

---

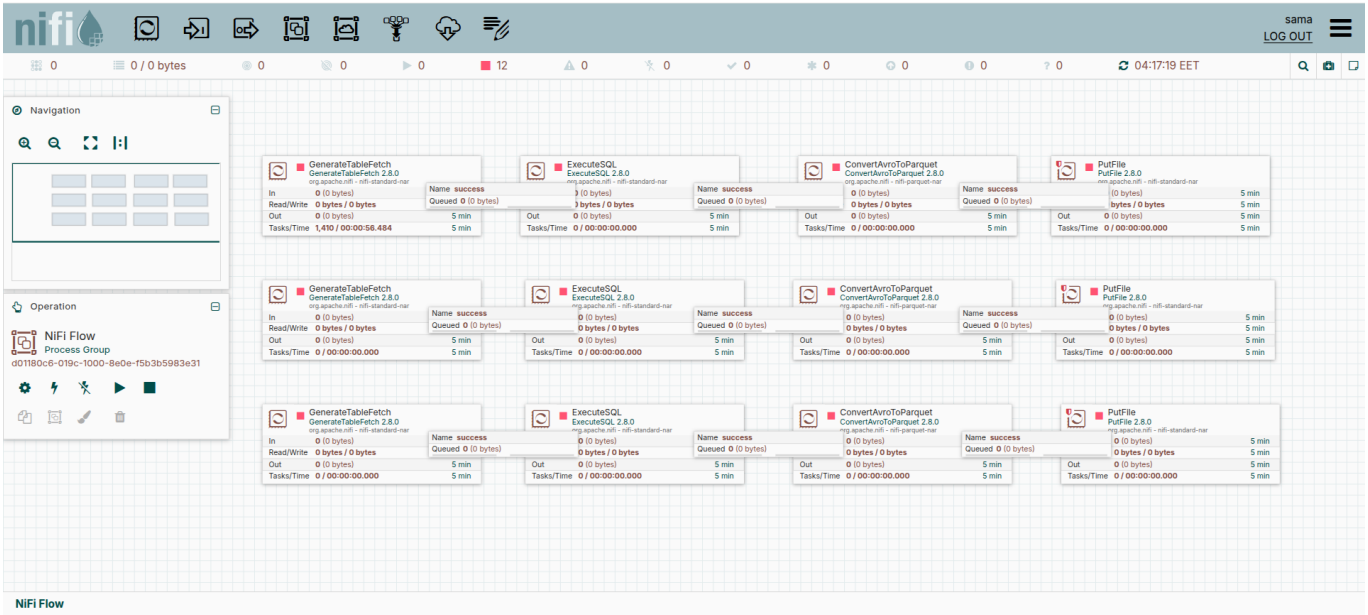
8. Generating Parquet Files with NiFi

NiFi processors are used to execute the ETL workflow. Prcoessors were selected to serve each stage of the ETL pipeline:

- **Extract**: ExecuteSQL processor takes the input data from the batch fetching output from the processor GenerateTableFetch, and executes SQL statements to read from the OLTP tables. The batching is vital to avoid OOM errors in NiFi.
- **Transform**: ConvertAvroToParquet takes the Avro-format data from ExecuteSQL and transforms it into a columnar format (Parquet).
- **Load**: PutFile and PutParquet ensure that parquet data is saved to a local file system and is persisted in the same format for later processing.

Typical pipeline:

```
GenerateTableFetch
      ↓
ExecuteSQL
      ↓
ConvertAvroToParquet
      ↓
PutFile
      ↓
PutParquet
```



Output files generated:

```
customer.parquet
supplier.parquet
lineitem.parquet
```

These files represent the analytical dataset.

---

9. Installing the Analytical Engine

To run analytical queries efficiently, Apache Spark was installed.

Spark provides distributed data processing and supports Parquet natively.

Spark Installation

Downloaded version:

```
Spark 3.5.8
```

Package type:

```
Pre-built for Hadoop 3.3+
```

Extracted directory example:

```
~/spark-3.5.8-bin-hadoop3
```

---

10. Verifying Spark Installation

Spark was tested using:

```
spark-shell
```

Expected output:

```
Spark session available as 'spark'
```

This confirmed that Spark was installed correctly.

---

11. Installing PySpark

To run Spark using Python, PySpark was installed.

Installation command:

```
pip install pyspark
```

---

12. Testing PySpark

A simple script was used to verify PySpark functionality.

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("Test").getOrCreate()

spark.range(10).show()
```

Example output:

```

```

```
+---+
| id|
+---+
| 0 |
| 1 |
| 2 |
| 3 |
| 4 |
| 5 |
| 6 |
| 7 |
| 8 |
| 9 |
+---+
```

This confirmed successful integration between:

- Python
- Spark
- Java
- PySpark

---

13. Preparing Analytical Queries

Once Parquet files are available, OLAP queries can be executed using Spark.

Typical workflow:

```
Read Parquet → Join Dimensions → Aggregate Results
```

Example PySpark code:

```
lineitem = spark.read.parquet("lineitem.parquet")
supplier = spark.read.parquet("supplier.parquet")
nation = spark.read.parquet("nation.parquet")

result = lineitem.join(supplier, "suppkey") \
                 .join(nation, "nationkey") \
                 .groupBy("n_name","s_name") \
                 .agg({"l_quantity":"sum"})
```

Query we used:

```
        SELECT
            c.N_NAME as nation,
            s.S_NAME as supplier,
            SUM(CAST(f.QUANTITY AS DOUBLE)) as sum_qty,
            SUM(CAST(f.EXTENDEDPRICE AS DOUBLE)) as sum_base_price,
            SUM(CAST(f.EXTENDEDPRICE AS DOUBLE) * (1 - CAST(f.DISCOUNT AS
DOUBLE))) as sum_disc_price,
            AVG(CAST(f.QUANTITY AS DOUBLE)) as avg_qty,
            COUNT(*) as count_order
        FROM
            fact f
            JOIN customer c ON f.C_CUSTKEY = c.C_CUSTKEY
            JOIN supplier s ON f.S_SUPPKEY = s.S_SUPPKEY
        WHERE
            f.SHIPDATE <= '1998-09-02'
        GROUP BY
            c.N_NAME, s.S_NAME
```

---

14. Final System Architecture

The full architecture of the system is:

```
TPCH OLTP Database (MySQL)
           ↓
      Apache NiFi
           ↓
     Parquet Files
           ↓
      Apache Spark
           ↓
       OLAP Query
```

This pipeline enables efficient large-scale analytical processing.

Results

```
ageCollectors or spark.eventLog.gcM
Run 1          | 14.9295
Run 2          | 5.5913
Run 3          | 4.3731
Run 4          | 4.2921
Run 5          | 4.2324
Run 6          | 4.1686
Run 7          | 3.7717
Run 8          | 3.8911
------------------------------
Average Spark Time: 5.6562 seconds
(base) mennamaggy@MennaMaggy:/media
```

FINAL COMPARISON

Run	OLTP (min)	OLAP (sec)
Run 1	2 min 50.94 sec	14.9295 sec
Run 2	3 min 11.49 sec	5.5913 sec
Run 3	2 min 39.52 sec	4.3731 sec
Run 4	2 min 55.83 sec	4.2921 sec
Run 5	2 min 42.87 sec	4.2324 sec
Run 6	3 min 44.34 sec	4.1686 sec
Run 7	3 min 28.06 sec	3.7717 sec
Run 8	3 min 50.30 sec	3.8911 sec
Average	3 min 10.42 sec	5.6562 sec